

Binary Heaps

Design and Analysis of Algorithms

Brian C. Dean

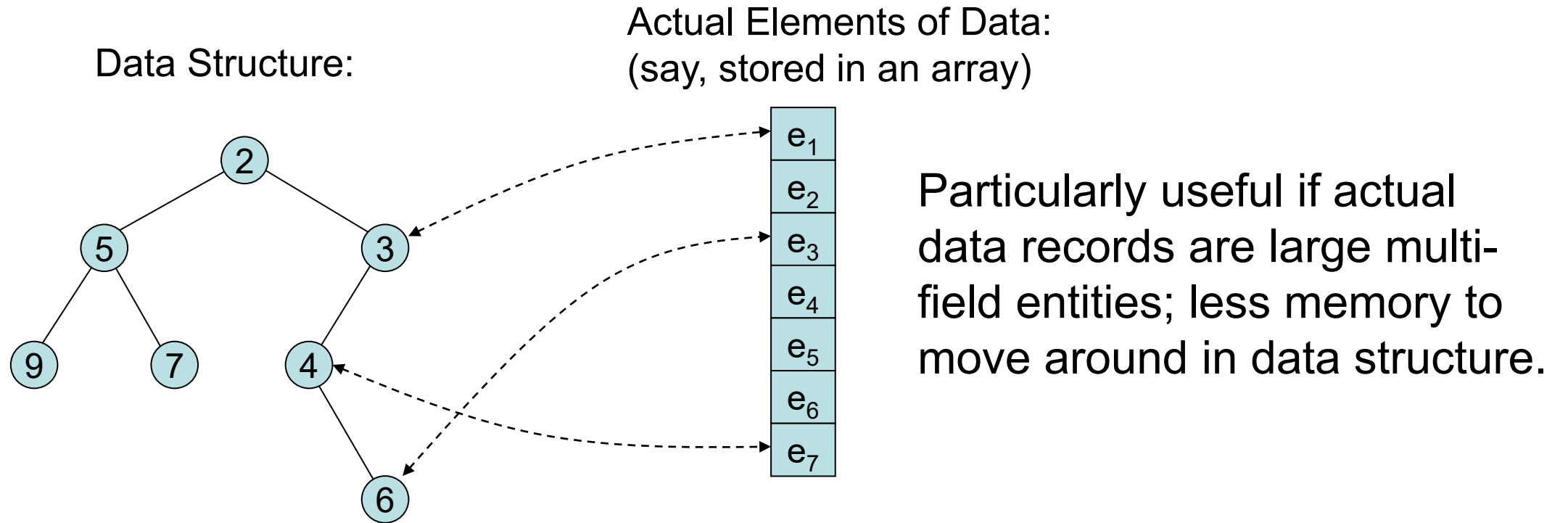
**School of Computing
Clemson University**



Recall: Priority Queues

- In a priority queue, the element with highest priority is always the first to exit.
- Main operations: *insert*, *remove-min* (or *remove-max*)
 - In a comparison-based setting, at least one of these must run in $\Omega(\log n)$ worst case time, due to $\Omega(n \log n)$ bound on comparison-based sorting.
- Other operations: *delete*, *increase-key*, *decrease-key*
 - Some redundancies here; e.g., *decrease-key* could be implemented by *delete* followed by *insert*.
- Notably not supported: ability to *find* an element once in structure.

Storing Elements Outside a Data Structure



Each record in the data structure keeps a pointer to the physical element of data it represents, and each element of data maintains a pointer to its corresponding record in the data structure, allowing you to find it “from the outside”.

Priority Queue Implementations

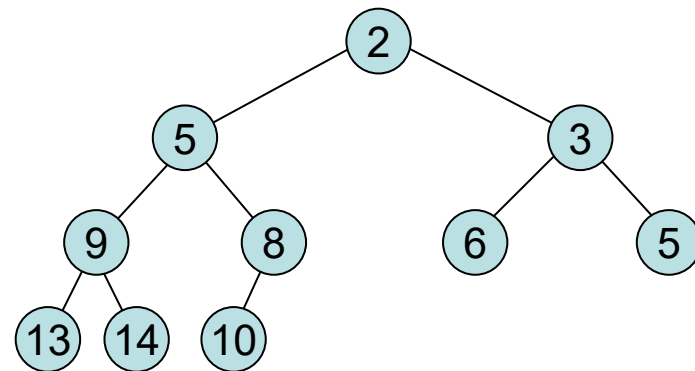
	<i>insert</i>	<i>remove-min</i>	<i>merge</i>
Unsorted array or linked list	$O(1)$	$\Theta(n)$	$\Theta(n)$
Sorted array or linked list	$\Theta(n)$	$O(1)$	$\Theta(n)$
Randomly mergeable binary heap (coded in module 2)	$O(\log n)$ in expectation	$O(\log n)$ in expectation	$O(\log n)$ in expectation
Binary heap (current discussion)	$O(\log n)$	$O(\log n)$	$\Theta(n)$
Skew heaps (next module)	$O(\log n)$ amortized	$O(\log n)$ amortized	$O(\log n)$ amortized
Balanced binary search tree (current module), skip lists and B-trees (next module)	$O(\log n)$ (possibly am or in exp)	$O(\log n)$ (possibly am or in exp)	$\Theta(n)$

The (Min-)Binary Heap

- An almost-complete binary tree (all levels full except the last, which is filled from the left side up to some point).
- Satisfies the **heap property**: for every element e , $\text{key}(\text{parent}(e)) \leq \text{key}(e)$.
 - Minimum element always resides at root.
- Physically stored in an array $A[1..n]$ with level-order encoding.
- Easy to move around the array in a treelike fashion:
 - $\text{Parent}(i) = \text{floor}(i/2)$.
 - $\text{Left-child}(i) = 2i$
 - $\text{Right-child}(i) = 2i + 1$.

	1	2	3	4	5	6	7	8	9	10
A:	2	5	3	9	8	6	5	13	14	10

Actual array representation in memory



Mental picture as a tree

Implementing Operations Using Sift-Up and Sift-Down

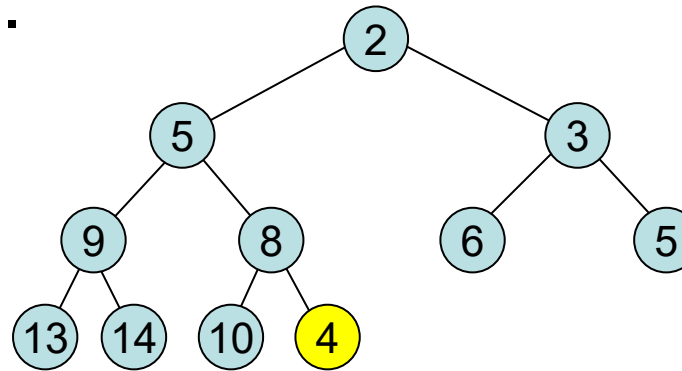
- All binary heap operations are built from the two fundamental operations:
 - *sift-up*(i) : As long as $A[i]$ violates the heap property with its parent, swap $A[i]$ with parent and update i accordingly.
("light elements float upward")
 - *sift-down*(i) : As long as $A[i]$ violates the heap property with one of its children, swap $A[i]$ with its smallest child and update i accordingly.
("heavy elements sink downward")
- Both take $O(\log n)$ time since the height of heap is $O(\log n)$.
- Elsewhere, *sift-down* sometimes called *heapify* or *down-heap*, the *sift-up* sometimes called *up-heap*.

Examples of Binary Heap Operations

- *Insert*: place new element (4, in this case) at end of heap array, then sift it up.

A:

1	2	3	4	5	6	7	8	9	10	11
2	5	3	9	8	6	5	13	14	10	4

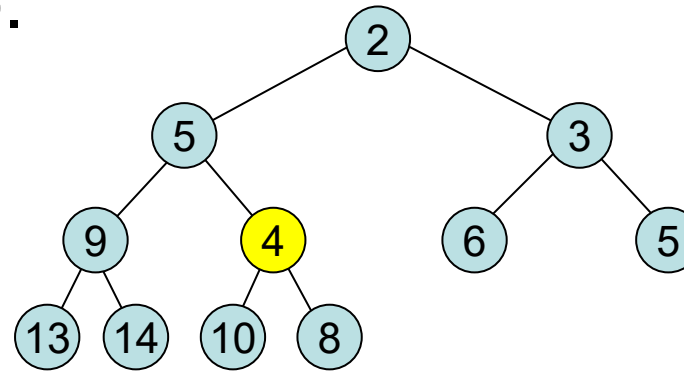


Examples of Binary Heap Operations

- *Insert*: place new element (4, in this case) at end of heap array, then sift it up.

A:

1	2	3	4	5	6	7	8	9	10	11
2	5	3	9	4	6	5	13	14	10	8

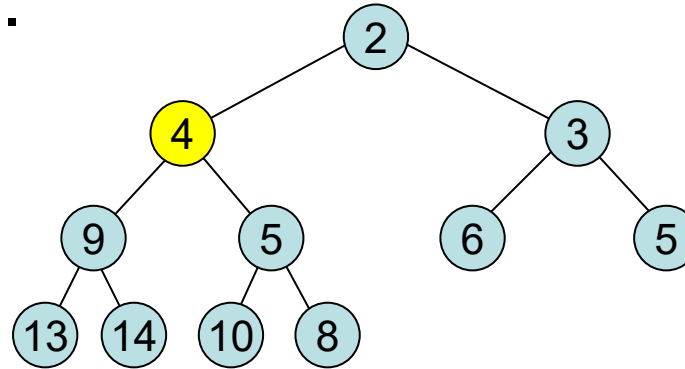


Examples of Binary Heap Operations

- *Insert*: place new element (4, in this case) at end of heap array, then sift it up.

A:

	1	2	3	4	5	6	7	8	9	10	11
	2	4	3	9	5	6	5	13	14	10	8

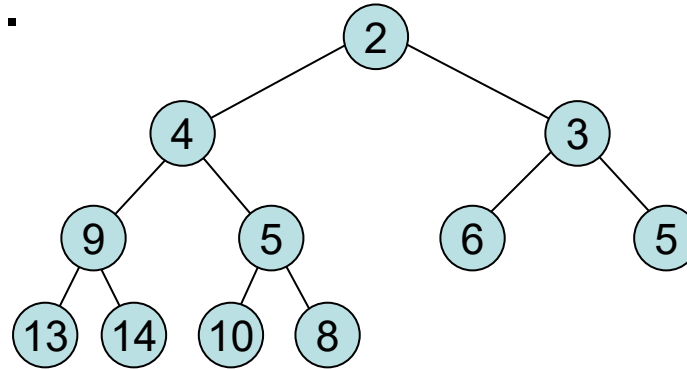


Examples of Binary Heap Operations

- *Insert*: place new element (4, in this case) at end of heap array, then sift it up.

A:

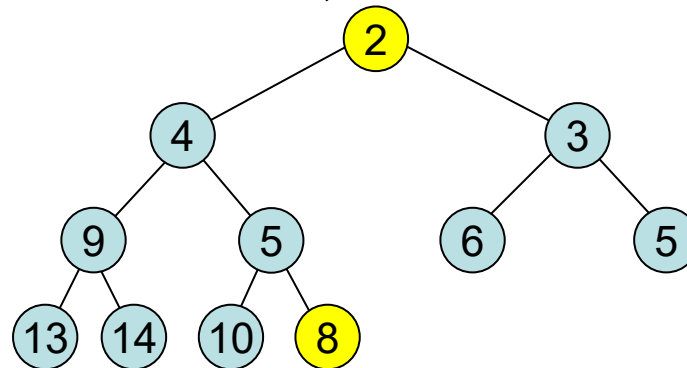
1	2	3	4	5	6	7	8	9	10	11
2	4	3	9	5	6	5	13	14	10	8



- *Remove-min*: swap elements 1 and n, then sift the root down.

A:

1	2	3	4	5	6	7	8	9	10	11
2	4	3	9	5	6	5	13	14	10	8

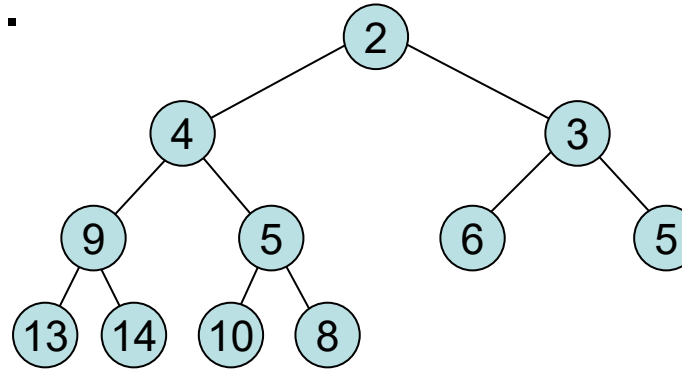


Examples of Binary Heap Operations

- *Insert*: place new element (4, in this case) at end of heap array, then sift it up.

A:

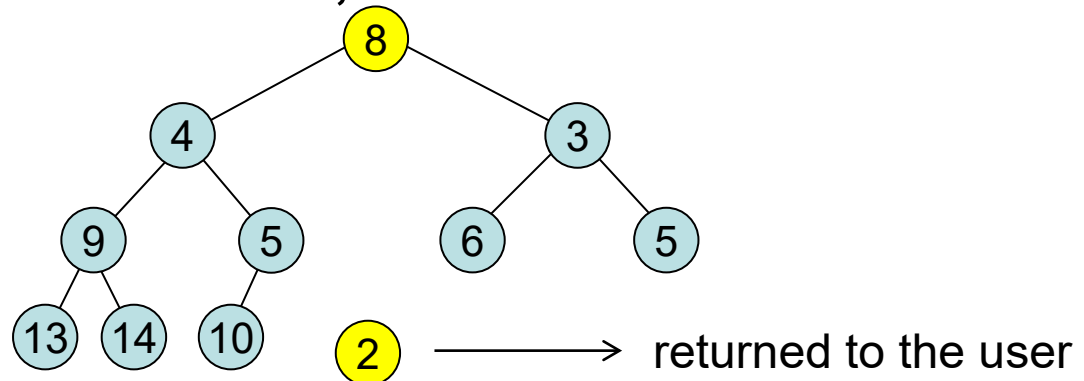
1	2	3	4	5	6	7	8	9	10	11
2	4	3	9	5	6	5	13	14	10	8



- *Remove-min*: swap elements 1 and n, then sift the root down.

A:

1	2	3	4	5	6	7	8	9	10
8	4	3	9	5	6	5	13	14	10

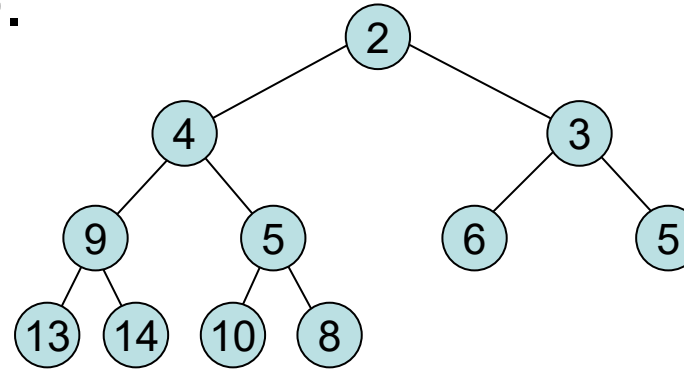


Examples of Binary Heap Operations

- *Insert*: place new element (4, in this case) at end of heap array, then sift it up.

A:

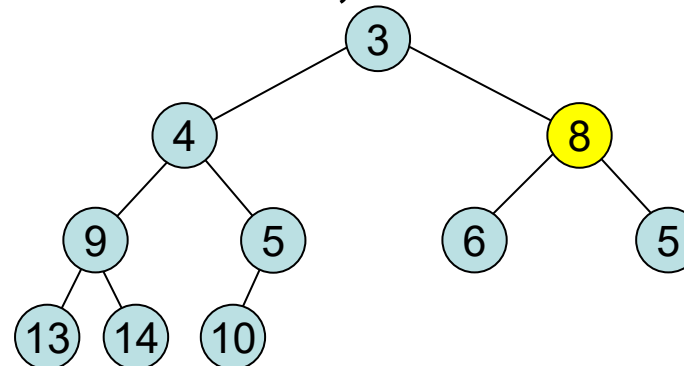
1	2	3	4	5	6	7	8	9	10	11
2	4	3	9	5	6	5	13	14	10	8



- *Remove-min*: swap elements 1 and n, then sift the root down.

A:

1	2	3	4	5	6	7	8	9	10
3	4	8	9	5	6	5	13	14	10

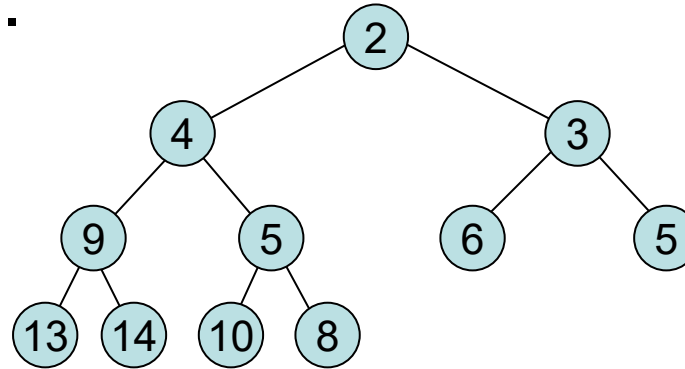


Examples of Binary Heap Operations

- *Insert*: place new element (4, in this case) at end of heap array, then sift it up.

A:

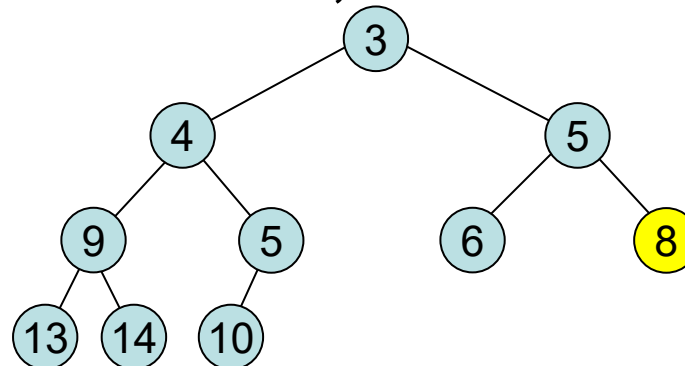
1	2	3	4	5	6	7	8	9	10	11
2	4	3	9	5	6	5	13	14	10	8



- *Remove-min*: swap elements 1 and n, then sift the root down.

A:

1	2	3	4	5	6	7	8	9	10
3	4	5	9	5	6	8	13	14	10



All Operations Easy to Implement Using Sift-Up and Sift-Down

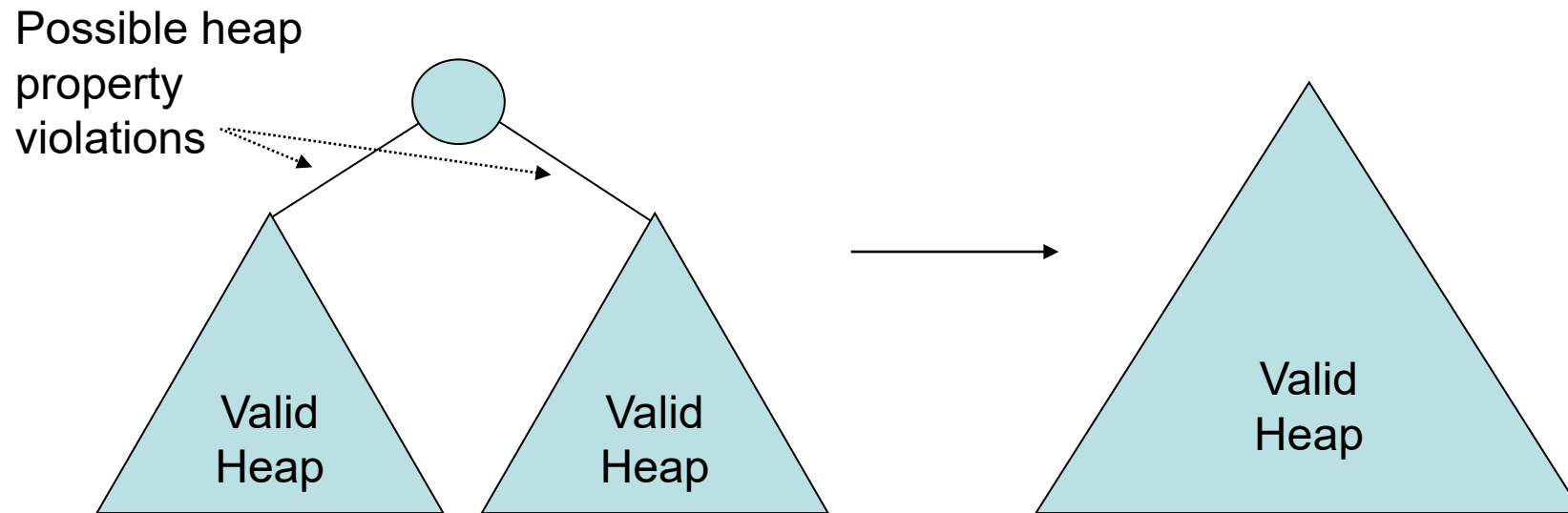
- General idea: modify the heap, then fix any violation of the heap property with calls to *sift-up* or *sift-down*.
 - *insert* : place new element in $A[n+1]$, then $\text{sift-up}(n+1)$.
 - *remove-min* : swap $A[n]$ and $A[1]$, then $\text{sift-down}(1)$.
 - *decrease-key*($i, \Delta k$) : decrease $A[i]$ by Δk , then $\text{sift-up}(i)$.
 - *increase-key*($i, \Delta k$) : increase $A[i]$ by Δk , then $\text{sift-down}(i)$.
 - *delete*(i) : swap $A[i]$ with $A[n]$, then $\text{sift-up}(i)$, $\text{sift-down}(i)$.
- All run in $O(\log n)$ time.

Building a Binary Heap

- We could build a binary heap in $O(n \log n)$ time using n successive calls to *insert*.
 - Equivalently: start with elements in arbitrary order in array, then call *sift-up*(i) for each i from $1 \dots n$.
 - Remarkable fact #1: if elements are inserted in random order, then this running time drops to $O(n)$ in expectation.
- Alternatively: start with elements in arbitrary order in array, then call *sift-down*(i) for each i from $n \dots 1$.
 - Remarkable fact #2: this builds a valid heap!
 - Remarkable fact #3: this runs in only $O(n)$ time!

Bottom-Up Binary Heap Construction

- The key property of *sift-down* is that it fixes an isolated violation of the heap property at the root:



- Using induction, it is now easy to prove that our “bottom-up” construction yields a valid heap.

Bottom-Up Binary Heap Construction: $O(n)$ Running Time

- Work per element is its height in the tree:
 - At most n elements in lowest level; 1 unit of work done to each by *sift-down*.
 - At most $n/2$ elements in 2nd lowest level; at most 2 units of work done to each.
 - At most $n/4$ elements in 3rd lowest level; at most 3 units of work done to each.
- So total time $\leq T = n + 2(n/2) + 3(n/4) + 4(n/8) + \dots$
(for simplicity, we carry the sum out to infinity, as this will certainly give us an upper bound).
- Claim: $T = 4n = O(n)$

Recall: “Shifting” Technique for Sums With a Geometric Character

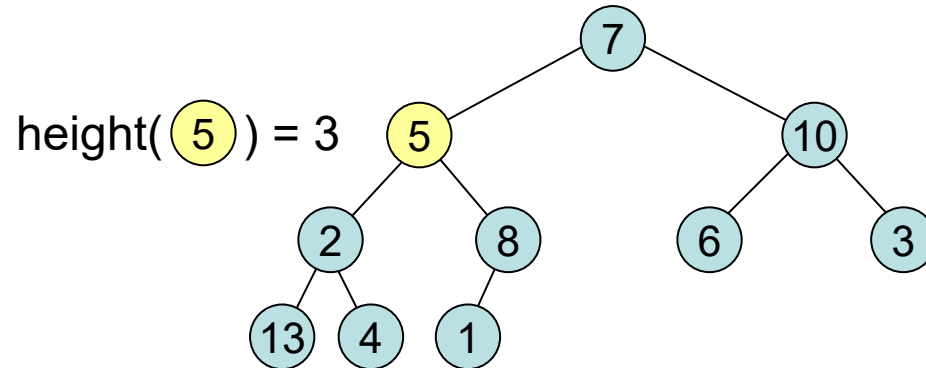
$$\begin{array}{rcl} T & = & n + 2(n/2) + 3(n/4) + 4(n/8) + \dots \\ - \quad T/2 & = & \quad n/2 + 2(n/4) + 3(n/8) + \dots \\ \hline T/2 & = & n + n/2 + n/4 + n/8 + \dots \end{array}$$

Applying the same trick again:

$$\begin{array}{rcl} T & = & 2n + n + (n/2) + (n/4) + \dots \\ - \quad T/2 & = & \quad n + (n/2) + (n/4) + \dots \\ \hline T/2 & = & 2n \end{array}$$

Building a Heap by Inserts in Random Order

- Recall: sum of heights = $O(n)$.
- How many expected changes at a given position in the heap, say at height h ?



- Only the elements in 5's subtree ($\leq 2^h$ of them) can sift up through this location and cause changes.
- As we successively sift these elements up, the value in the 5 position will be their running min. This is counting resets of a running min of a random ordering!
- $E[\text{resets}] = O(\log 2^h) = O(h)$

Heapsort

- Any priority queue can be used to sort.
- The binary heap gives us a particularly nice way to sort in $O(n \log n)$ time, known as **heapsort**:
 - Start with an array $A[1..n]$ of elements to sort.
 - Build a heap (e.g., with our bottom up method) on A in $O(n)$ time.
 - Call *remove-min* n times.
 - Afterwards, A will end up reverse-sorted (it would be forward-sorted if we had started with a “max” heap)
- Heapsort compares favorably to...
 - Merge sort, because heapsort runs **in place**.
 - Randomized quicksort, because heapsort is **deterministic**.